

# RT-NoDrop—实时任务调度框架设计与可调度性分析

## 摘要

泛在操作系统中的功能任务直接作用于物理环境和人类生活，工业物联网、智能边缘和机器人控制节点都需要同时运行控制、感知、通信和安全审计任务。系统审计、入侵检测、访问控制记录等安全任务并不只是离线日志处理，而是影响系统可信运行的实时负载。传统审计框架通常以旁路方式处理系统调用事件，调度器只看到功能任务本身，无法感知审计记录和审计消费带来的额外执行需求。当系统负载升高时，这种割裂设计会导致两类风险：功能任务响应时间失去可预测性，安全事件在内核缓冲区中长期滞留甚至丢失。

本文设计一个安全感知实时任务调度实验框架，将功能任务、系统调用事件和审计消费任务统一纳入固定优先级响应时间分析。该框架面向人机物融合系统中的跨层级协同调度：应用层功能任务产生安全事件，内核审计层记录和缓冲事件，调度分析层统一计算功能任务响应时间和安全事件驻留时间。框架复用 SchedCAT 生成周期实时任务集，比较基础 RTA、OmniLog、Audit、Ellipsis 和 NoDrop 等审计策略在不同 CPU 利用率、系统调用强度和 CPU 数量下的可调度率，并进一步给出安全事件驻留时间的上界统计。

## 1. 引言

在人机物融合场景中，调度器需要同时处理控制、感知、通信和安全审计等多类任务。以工业边缘控制节点为例，控制回路和传感数据处理任务决定实时响应，系统调用审计和访问行为记录决定安全可追溯性，两类任务共享 CPU、内核缓冲区和调度优先级空间。安全审计任务并不是传统意义上的后台任务；它们记录系统调用、文件访问和进程行为，是系统安全性的组成部分。如果审计事件无法及时消费，系统虽然可能仍完成了功能任务，却无法保证安全事件完整性。相反，如果把审计开销保守地全部加入功能任务执行时间，又会导致过度资源预留和可调度率下降。

因此，安全感知调度需要同时满足两个目标。第一，功能任务在引入审计机制后仍满足 deadline。第二，审计事件从生成到被消费的驻留时间具有可计算上界。本文框架围绕这两个目标建立统一模型：系统调用跟踪开销作为功能任务 WCET 的扩展项，审计消费过程作为显式 consumer task 参与调度分析。通过把功能任务和安全任务置于同一可调度性分析中，框架把原本割裂的应用执行、内核审计和调度决策融合为一个轻量化实验模型，为多核边缘节点上的安全任务混布和资源利用率评估提供基础。

## 2. 系统模型

系统由  $N$  个周期或 sporadic 实时任务组成，记为

$$\tau = \{\tau_1, \tau_2, \dots, \tau_N\}. \quad (1)$$

任务运行在  $M$  个处理器核心上，采用分区固定优先级调度。每个任务  $\tau_i$  被静态分配到某个核心，并用三元组表示：

$$\tau_i = (e_i, p_i, s_i). \quad (2)$$

其中  $e_i$  是任务原始最坏执行时间， $p_i$  是最小到达间隔。实验中采用隐式 deadline 任务模型，即任务 deadline 等于周期  $p_i$ 。 $s_i$  表示任务每个 job 最多触发的系统调用次数。由于本文关注硬实时场景，假设每个任务的系统调用次数和每次系统调用的审计开销都有上界。

表 1 给出分析中使用的主要符号。

| 符号           | 含义                                   | 实现映射                                                       |
|--------------|--------------------------------------|------------------------------------------------------------|
| $\tau_i$     | 第 $i$ 个功能任务                          | <code>is_consumer=False</code> 的 <code>SporadicTask</code> |
| $\sigma_i$   | 由 $\tau_i$ 派生的审计消费任务                 | <code>is_consumer=True</code> 的 consumer task              |
| $e_i$        | 功能任务原始 WCET                          | 任务生成阶段的 <code>task.cost</code>                             |
| $E_i$        | 加入 syscall 跟踪开销后的 WCET               | <code>AuditFramework.calculate_execution_time()</code>     |
| $p_i$        | 功能任务周期和 deadline                     | <code>task.period</code>                                   |
| $s_i$        | 每个 job 的最大 syscall 次数                | <code>task.syscall_count</code>                            |
| $r_i$        | 功能任务响应时间上界                           | RTA 迭代中的 <code>task.response_time</code>                   |
| $b_i$        | 功能任务阻塞时间                             | <code>prio_inversion</code> ，默认 0                          |
| $hp_i$       | 与 $\tau_i$ 同核且优先级高于 $\tau_i$ 的功能任务集合 | 按 <code>preemption_level</code> 排序得到                       |
| $chp_i$      | 优先级高于 $\tau_i$ 的 consumer 集合         | NoDrop 分析中的 higher-priority consumers                      |
| $\Delta$     | 每次 syscall 的审计记录开销上界                 | <code>params.py</code> 中的 <code>delta</code>               |
| $\alpha$     | consumer 每次调用的固定开销                   | <code>params.py</code> 中的 <code>alpha</code>               |
| $\beta$      | consumer 处理单个 syscall 事件的开销          | <code>params.py</code> 中的 <code>beta</code>                |
| $q_i$        | consumer $\sigma_i$ 的调用周期            | consumer task 的 <code>period</code>                        |
| $\epsilon_i$ | consumer $\sigma_i$ 的单次执行 WCET       | 由 $\alpha, \beta, q_i, p_i, s_i$ 计算                        |
| $R_i$        | $\tau_i$ 产生的审计事件驻留时间上界               | <code>RES_MAX_ND</code> 等统计列的基础值                           |
| $T_i^w$      | 事件在内核缓冲区等待 consumer 的时间              | residence-time waiting term                                |
| $T_i^p$      | 事件在 consumer 内完成处理的时间                | residence-time processing term                             |

审计框架引入两类额外开销。第一类是每次系统调用触发的记录开销  $\Delta$ ，该开销发生在功能任务上下文中，会直接延长功能任务执行时间。第二类是 consumer 处理审计事件时的执行开销，包括固定调用成本  $\alpha$  和按事件数量增长的处理成本  $\beta$ 。

### 3. 框架设计

#### 3.1 总体结构

框架由任务生成、审计开销建模、响应时间分析、事件驻留时间分析和实验编排五个部分组成。任务生成模块基于 SchedCAT EMSTADA 生成不同利用率下的周期任务集，并为每个任务附加 syscall 负载。审计开销建模模块在 `rta/params.py` 中维护不同审计框架的  $\Delta$ 、 $\alpha$ 、 $\beta$  参数。响应时间分析模块包括基础 RTA、OmniLog、Audit、Ellipsis 和 NoDrop 分析。事件驻留时间分析模块只在 NoDrop 可调度样本上统计  $R_i$ 。实验编排入口 `rta/safety_aware_sched.py` 提供可复现 CLI，固定随机种子并输出完整配置、环境和数据。

从多级融合调度角度看，框架包含三层协同关系。功能任务层描述控制、感知和通信任务的周期、执行时间与 syscall 强度；安全审计层描述 syscall trace、事件缓冲和 consumer 处理开销；调度分析层把两类任务统一映射为固定优先级任务系统，计算 deadline 可满足性和事件驻留时间。三层之间通过  $s_i$ 、 $\Delta$ 、 $\alpha$ 、 $\beta$  和  $q_i$  等参数连接，避免把安全任务作为调度器不可见的外部负载。

## 3.2 任务与 consumer 生成

对每个 CPU 分区，实验生成  $n$  个功能任务，并按照周期设置固定优先级。短周期任务具有更高优先级。每个功能任务  $\tau_i$  关联 syscall 负载  $s_i$ 。在安全感知策略中，系统为功能任务派生 consumer  $\sigma_i$ ，用于周期性处理该任务产生的审计事件。默认复现实验取  $q_i = p_i$ ，即 consumer 周期与其对应功能任务周期一致。

这种建模方式避免把审计消费过程隐藏在不可见后台线程中。consumer 与功能任务共同参与优先级排序，因此功能任务响应时间可以显式包含审计任务抢占干扰，安全事件驻留时间也可以被单独界定。

## 3.3 对照框架

基础 RTA 不考虑审计任务，用作功能任务基线。OmniLog、Audit 和 Ellipsis 代表现有单 consumer 审计框架，它们用一个全局 consumer 处理所有任务产生的事件。NoDrop 代表安全感知策略，它将 consumer 与功能任务绑定，以更细粒度的周期处理审计事件。实验同时输出这些策略的可调度率，用于刻画安全审计设计对实时性的影响。

# 4. 响应时间分析

## 4.1 基础 RTA

在不启用审计机制时，任务  $\tau_i$  的响应时间由经典固定优先级 RTA 计算。响应时间  $r_i$  是下式的最小正不动点：

$$r_i = e_i + b_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{r_i}{p_k} \right\rceil e_k. \quad (3)$$

求解过程从任务自身执行时间、阻塞时间和高优先级任务的一次执行需求开始，随后在每轮迭代中根据当前窗口长度重新计算高优先级任务释放次数。当新旧需求相等时迭代收敛；若收敛值不超过  $p_i$ ，任务满足 deadline。若需求超过  $p_i$  仍无法收敛，则该任务不可调度。

代码中 `rta/rta.py` 的 `bound_response_times()` 实现该分析。实验输出中的 `#rta` 列即为不引入审计 consumer 时的可调度率。

## 4.2 syscall 跟踪开销

审计框架在每次 syscall 上增加记录开销。若任务  $\tau_i$  每个 job 最多触发  $s_i$  次 syscall，单次 syscall 最大开销为  $\Delta$ ，则加入审计记录后的扩展 WCET 为

$$E_i = e_i + s_i \Delta. \quad (4)$$

后续所有审计感知分析都使用  $E_i$  替代基础 RTA 中的  $e_i$ 。在实现中，该转换由 `rta/params.py` 中的 `AuditFramework.calculate_execution_time()` 完成。不同审计框架通过不同的  $\Delta$ 、 $\alpha$  和  $\beta$  参数实例化。

## 4.3 NoDrop 多 consumer 响应时间

NoDrop 风格的安全感知调度显式建模每个 consumer 的执行需求。consumer  $\sigma_i$  每次调用需要完成固定转移和上下文切换成本  $\alpha$ ，并处理当前周期内由  $\tau_i$  产生的审计事件。周期  $q_i$  内  $\tau_i$  最多释放  $\lceil q_i/p_i \rceil$  个 job，因此 consumer  $\sigma_i$  的单次执行 WCET 为

$$\epsilon_i = \alpha + \left\lceil \frac{q_i}{p_i} \right\rceil s_i \beta. \quad (5)$$

设  $chp_i$  为所有优先级高于  $\tau_i$  的 consumer 集合。consumer 对  $\tau_i$  的抢占干扰为

$$I(\sigma_i) = \sum_{\sigma_k \in chp_i} \left\lceil \frac{r_i}{q_k} \right\rceil \epsilon_k. \quad (6)$$

功能任务  $\tau_i$  在 NoDrop 下的完整响应时间满足

$$r_i = E_i + b_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{r_i}{p_k} \right\rceil E_k + I(\sigma_i). \quad (7)$$

该公式保留基础 RTA 的高优先级功能任务干扰，并额外加入高优先级 consumer 的审计处理干扰。任务集可调度当且仅当所有功能任务和参与分析的 consumer 都能在各自 deadline 内完成。代码中 `rta/rta_nodrop.py` 的 `bound_response_times_nodrop()` 和 `_rta_nodrop()` 实现该分析，实验输出中的 `#rta_nodrop` 列统计其可调度率。

## 5. 事件驻留时间分析

响应时间分析保证功能任务 deadline，事件驻留时间分析保证安全事件处理及时性。对任务  $\tau_i$  产生的审计事件，驻留时间  $R_i$  由两部分组成：事件在内核缓冲区等待 consumer 调用的时间  $T_i^w$ ，以及事件进入 consumer 后完成处理的时间  $T_i^p$ 。

$$R_i = T_i^w + T_i^p. \quad (8)$$

在周期性 consumer 设计中，最坏情况是事件刚错过一次 consumer 调用，因此等待时间由 consumer 周期界定：

$$T_i^w = q_i. \quad (9)$$

处理时间  $T_i^p$  不能简单取  $\alpha + s_i\beta$ ，因为 consumer 自身也会被高优先级功能任务和高优先级 consumer 抢占。因此  $T_i^p$  是 consumer 响应时间的不动点上界：

$$T_i^p = \epsilon_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{T_i^p}{p_k} \right\rceil E_k + \sum_{\sigma_k \in chp_i \setminus \{\sigma_i\}} \left\lceil \frac{T_i^p}{q_k} \right\rceil \epsilon_k. \quad (10)$$

于是事件驻留时间上界为

$$R_i = q_i + T_i^p. \quad (11)$$

实验入口已经输出该指标。在 `rta/safety_aware_sched.py` 中，只有当 `#rta_nodrop` 对当前样本判定可调度时，框架才调用 `collect_nodrop_residence_times()`。该函数遍历每个 CPU 分区和每个功能任务，调用 `rta/rta_nodrop.py` 中的 `_event_residence_time_nodrop()` 计算  $R_i$ ，最后汇总为 `RES_MAX_ND`、`RES_MIN_ND` 和 `RES_MEAN_ND`。其中 `RES_MAX_ND` 表示可调度样本中的最坏事件驻留时间，`RES_MEAN_ND` 表示平均安全事件处理延迟。

当前复现实验默认  $q_i = p_i$ ，因此代码中 `_event_residence_time_nodrop()` 使用 `T_w = task.period` 与上述公式一致。如果后续启用 consumer period optimization，则实现应显式记录每个 consumer 的  $q_i$ ，并以  $T_i^w = q_i$  代替默认周期。

## 6. consumer 周期选择

consumer 周期  $q_i$  同时影响功能任务可调度性和事件驻留时间。较小的  $q_i$  会缩短事件等待时间，但会增加 consumer 激活频率和抢占干扰；较大的  $q_i$  会降低调度干扰，但会延长安全事件在 buffer 中的等待时间。因此周期选择可以形式化为带可调度性约束的驻留时间优化问题：

$$\begin{aligned} \min_{\mathbf{q}} \quad & \sum_i R_i(\mathbf{q}) \\ \text{s.t.} \quad & \forall \tau_i, r_i(\mathbf{q}) \leq p_i, \\ & q_i^{\min} \leq q_i \leq q_i^{\max}. \end{aligned} \quad (12)$$

使用粒子群优化 PSO 搜索周期向量  $\mathbf{q} = \{q_1, q_2, \dots, q_n\}$ 。若某个周期向量导致任务集不可调度，则其 fitness 设为  $+\infty$ ；否则 fitness 取事件驻留时间总和。粒子速度和位置更新为

$$v(t+1) = wv(t) + c_1 r_1 (p_{\text{best}} - q(t)) + c_2 r_2 (g_{\text{best}} - q(t)), \quad (13)$$

$$q(t+1) = \text{clip}(\text{round}(q(t) + v(t+1)), q_{\min}, q_{\max}). \quad (14)$$

其中  $p_{\text{best}}$  是粒子历史最优位置， $g_{\text{best}}$  是群体全局最优位置， $r_1$  和  $r_2$  是均匀随机数。论文给出的搜索范围以功能任务周期为下界，并用 deadline 约束确定上界：

$$q_i^{\min} = p_i, \quad (15)$$

$$E_i + \left\lceil \frac{q_i^{\max}}{p_i} \right\rceil s_i \beta = p_i. \quad (16)$$

当前仓库的复现实验选择固定  $q_i = p_i$ ，优先保证 Fig. 4 风格实验稳定复现。PSO 周期选择可作为后续扩展接入，形成新的 NoDrop 优化版本，并与固定周期版本比较驻留时间收益。

## 7. 现有单 consumer 审计框架分析

OmniLog、Audit 和 Ellipsis 等现有框架通常使用一个全局 consumer  $\sigma$  处理所有任务产生的审计事件。为了给出可比较的乐观上界，分析假设该 consumer 周期性运行并具有较高优先级，同时忽略部分固定唤醒开销。关键是先计算任意窗口  $W$  内系统可能产生的 syscall 数量。

任务  $\tau_i$  在长度为  $W$  的窗口内最多释放

$$\left\lceil \frac{W + r_i}{p_i} \right\rceil \quad (17)$$

个 job，因此其 syscall 数量上界为

$$S_i(W) = \left\lceil \frac{W + r_i}{p_i} \right\rceil s_i. \quad (18)$$

系统总 syscall 数量上界为

$$S(W) = \sum_{i=1}^N S_i(W). \quad (19)$$

若全局 consumer 的周期为  $q$ ，consumer 自身响应时间为  $r_\sigma$ ，则一次 consumer 执行最多处理的 syscall 数为

$$A^* = S(q + r_\sigma) = \sum_{i=1}^N \left\lceil \frac{q + r_\sigma + r_i}{p_i} \right\rceil s_i. \quad (20)$$

在忽略固定调用开销  $\alpha$  的乐观情况下, consumer 单次执行 WCET 为  $A^*\beta$ 。功能任务  $\tau_i$  的响应时间满足

$$r_i = E_i + b_i + \sum_{\tau_k \in hp_i} \left\lceil \frac{r_i}{p_k} \right\rceil E_k + \left\lceil \frac{r_i}{q} \right\rceil A^*\beta. \quad (21)$$

该分析对应 `rta/rta_omnilog.py` 的单 consumer 逻辑, 以及 `rta/util_syscall_cpu.py` 中 `#rta_omnilog_delta=*`、`#rta_audit`、`#rta_ellipsis` 等对照列。它体现了现有框架的结构性问题: 即使采用乐观假设, 集中 consumer 仍会把所有任务产生的安全事件聚合到同一个执行实体中, 导致响应时间干扰和优先级反转难以控制。

## 8. 实现与复现

实验入口为 `rta/safety_aware_sched.py`。该文件不重新实现分析公式, 具体运行方式如下:

```
git submodule update --init --recursive
source setpath.sh
cd lib/schedcat && make && cd ../../
python rta/safety_aware_sched.py --experiment utilization --samples 1000
```

实验支持三类扫描。`utilization` 扫描 CPU 利用率, 用于观察系统在高资源占用下保持实时响应的能力。`syscall_count` 固定利用率并改变 syscall 负载, 用于刻画安全事件强度增加时审计框架对调度延迟和事件处理延迟的影响。`cpu_count` 改变 CPU 数量, 用于观察多分区部署下功能任务和安全任务混布时的扩展趋势。输出文件位于 `output/safety_aware_sched/`, 每个文件包含配置、运行环境、数据表和运行耗时。

输出数据的核心列包括各策略可调度率和 NoDrop 事件驻留时间统计。可调度率反映功能任务 deadline 是否可保证, 可用于衡量实时响应能力; `utilization` 维度反映资源利用率变化; `cpu_count` 维度反映多核或多设备分区部署下的扩展性; `RES_MAX_ND`、`RES_MIN_ND` 和 `RES_MEAN_ND` 反映安全事件处理完整性和及时性。两类指标共同刻画功能调度性和安全任务完整性之间的权衡。

## 9. 兼容性与边界条件

仓库沿用原始 SchedCAT 实验环境, 推荐 Python 2.7、NumPy、SciPy、GNU Make、SWIG、g++ 和 libgmp。当前本地入口增加了延迟导入逻辑, 因此 `--help` 不依赖 SchedCAT; 真正运行实验时仍需要初始化并构建 `lib/schedcat`。若依赖缺失, 入口会给出子模块初始化和构建命令。

分析模型默认采用分区固定优先级调度、隐式 deadline、每个 job 的 syscall 次数有界、每次 syscall 审计开销有界。若迁移到 OpenHarmony 内核或实际设备, 可以把本文的 consumer task 模型映射为安全审计服务线程、内核审计任务或调度类中的周期性安全任务, 并把  $q_i$ 、优先级和 CPU 绑定关系作为调度参数暴露给系统。平台迁移时需要测量对应设备上的  $\Delta$ 、 $\alpha$  和  $\beta$ , 再用本文分析判断功能任务 deadline 与审计事件驻留时间是否同时满足。如果系统采用动态优先级、共享资源锁或跨核迁移, 还需要在当前 RTA 基础上加入相应阻塞和迁移开销项。